

**Nivel****Nombre curso****No. Guía**

Universitario

Arquitectura de Soluciones de
Software

Taller #1

Taller #1: Clean Architecture aplicada con SOLID y Clean Code**Sistema de préstamo de equipos del laboratorio**

Objetivo: Aplicar Clean Architecture, los principios SOLID y las prácticas de Clean Code para diseñar y construir el núcleo de un sistema de préstamo de equipos con persistencia en SQLite, documentando el diseño con diagramas de clases y de secuencia.

Instrucciones del taller: Trabajo en grupos de máximo 3 personas. El laboratorio de la facultad presta equipos a los estudiantes, pero hoy lo hace en una hoja de cálculo: se prestan equipos dañados, hay estudiantes con demasiados equipos y las multas por retraso se calculan a mano. Deben construir el núcleo del sistema con dos casos de uso, registrar préstamo y registrar devolución, y entregar los siguientes elementos.

Parte 1: Reglas de negocio

El sistema debe cumplir las siguientes reglas:

- **R1:** Un estudiante puede tener como máximo 2 préstamos activos.
- **R2:** Solo se presta un equipo en estado DISPONIBLE.
- **R3:** El plazo depende de la categoría: PORTATIL 3 días, CAMARA 2 días y KIT_ROBOTICA 1 día.
- **R4:** Un estudiante con multa pendiente no puede pedir prestado.
- **R5:** Si la devolución supera la fecha límite, la multa es (días completos de retraso) por la tarifa diaria de la categoría: PORTATIL \$5.000, CAMARA \$8.000 y KIT_ROBOTICA \$12.000. Devolver el mismo día de la fecha límite no genera multa.
- **R6:** Si el equipo se devuelve con daño, pasa a EN_MANTENIMIENTO; si no, pasa a DISPONIBLE.
- **R7:** Se notifica al estudiante (simulado) al prestar (indicando la fecha límite) y al devolver con multa.

Parte 2: Requisitos técnicos

El proyecto debe cumplir con lo siguiente:

- **Casos de uso:** deben llamarse **RegistrarPréstamo** y **RegistrarDevolucion**.
- **Entidades mínimas:** **Estudiante**, **Equipo** y **Préstamo**.
- **Puertos mínimos:** los repositorios (estudiantes, equipos y préstamos), **Notificador** y **ProveedorFecha** (entrega la fecha actual mediante un método hoy()).
- **Persistencia:** adaptadores **SQLite** reales con las tablas estudiantes, equipos y préstamos. Además, una implementación en memoria (Con listas) del repositorio de préstamos, para demostrar que puede sustituirse.
- **Fecha:** una implementación que use la fecha real del sistema y otra de fecha fija, que se usa en el demo.
- **Ejecución:** main.py con un modo demo que ejecute los casos de aceptación con la fecha fija. Se debe usar solo la biblioteca estándar de Python.
- **Convención del curso:** una clase por archivo.

- **Estructura de carpetas:** dominio/, aplicacion/ (con puertos/ y casos_uso/), infraestructura/, docs/ y main.py en la raíz.

Parte 3: Qué deben demostrar en su proyecto

Para cada tema debe existir evidencia verificable:

- **SRP:** la mora, la notificación y la persistencia no están mezcladas con las reglas de préstamo.
- **OCP:** agregar la categoría PROYECTOR (2 días, \$6.000 por día) solo añade archivos y una línea de composición; no se modifica ningún archivo del dominio ni de los casos de uso. No se aceptan cadenas de if/elif por categoría.
- **LSP:** cambiando una sola línea de main.py, el demo produce el mismo resultado con el repositorio de préstamos SQLite y con el de memoria.
- **ISP:** Cada puerto corresponde a una responsabilidad. No se acepta un único RepositorioGeneral simultaneo para estudiantes, equipos y préstamos, ni un puerto que mezcle persistencia con notificación o con la fecha.
- **DIP:** los casos de uso dependen solo de entidades y puertos. main.py es el único que conoce las clases concretas.
- **Clean Code:** nombres que expresan intención, funciones de máximo 15 líneas, sin índices mágicos (plazos y tarifas con nombre), excepciones de dominio con nombre propio y sin parámetros booleanos ambiguos.
- **Clean Architecture:** revisión de las importaciones de cada archivo.
 - Ningún archivo de dominio/ importa sqlite3, aplicación ni infraestructura.
 - Ningún archivo de aplicación/ importa sqlite3 ni infraestructura.
 - Solo main.py importa clases de infraestructura.
 - Además, deben listar en docs/JUSTIFICACION.md los imports de una clase de cada capa y explicar por qué cumplen la regla.

Parte 4: Casos de aceptación (con fecha fija: 2026-10-05)

El demo debe reproducir los siguientes escenarios:

- **CA1:** Ana no tiene préstamos y pide PORTATIL-01. Se crea el préstamo con fecha límite 2026-10-08 y se envía la notificación.
- **CA2:** Ana ya tiene 2 préstamos activos y pide un tercer equipo. Se rechaza por límite de préstamos.
- **CA3:** CAMARA-02 fue prestada el 2026-10-01 (límite 2026-10-03) y se devuelve el 2026-10-06. Se genera una multa de 3 días por \$8.000 = \$24.000 y se notifica.
- **CA4:** Luis tiene una multa pendiente y pide un equipo. Se rechaza por multa pendiente.
- **CA5:** Un equipo se devuelve con daño. Queda EN_MANTENIMIENTO y no se puede prestar.
- **CA6:** Se agrega la categoría PROYECTOR. Debe funcionar sin modificar el dominio ni los casos de uso.

Nota: para reproducir los escenarios, el demo debe cargar datos iniciales en SQLite (por ejemplo, Ana con dos préstamos activos y Luis con una multa pendiente).

Entregables

Cada pareja debe entregar:

1. **Diagrama de clases:** Imagen PNG que muestre las cuatro capas, los puertos y el sentido de las dependencias.

2. **Diagramas de secuencia:** tres escenarios: préstamo exitoso, préstamo rechazado por una regla (a elección) y devolución tardía con multa. Las líneas de vida deben nombrar objetos concretos, no interfaces.
3. **Proyecto funcional:** un archivo .zip que se ejecute con python main.py.
4. **Justificación:** archivo docs/JUSTIFICACION.md (máximo una página) con una tabla de "principio, archivo y decisión concreta donde se aplica", la revisión de importaciones y la declaración de uso de IA.

Uso de inteligencia artificial

- **No se permite:** usar herramientas de IA generativa (ChatGPT, Claude, Copilot, Gemini u otras) para producir el diseño, el código, los diagramas o la justificación, ni para completar, corregir o mejorar partes de ellos.
- **Sí se permite:** consultar la documentación oficial de Python y SQLite, los apuntes y las diapositivas del curso.
- **Declaración obligatoria:** docs/JUSTIFICACION.md debe cerrar con la frase "Declaramos que el diseño, el código y los diagramas son de nuestra autoría y que no usamos IA generativa para producirlos", seguida de los nombres de la pareja.
- **Sustentación:** cada integrante defiende el trabajo de forma oral e individual, explica cualquier archivo que se le señale y realiza en vivo un cambio pequeño (por ejemplo, agregar la categoría PROYECTOR o cambiar una tarifa) sin ayuda externa.
- **Consecuencia:** si no pueden explicar o modificar su propio código, la entrega se considera no autoral y se evalúa como no presentada.

Finalmente:

- Se entrega un único archivo .zip por equipo, con el proyecto y la carpeta docs/.
- La fecha máxima de envío es un día antes de la fecha pactada en clase para la sustentación, si esta se envía después de la hora máxima pactada, se omitirá y se entenderá que no presentaron el taller.
- **El uso de IA generativa en este taller no está permitido y se verifica en la sustentación oral.**